EE 66
Fall 2026

# Signals, Dynamics, and Information

# Homework 10

## This homework is due Friday, November 6, 2026, at 23:59.

**Submission Format**

Your homework submission should consist of a single PDF file that contains all of your answers (any handwritten answers should be scanned).

UCB EE 66, Fall 2026, Homework 10, All Rights Reserved. This may not be publicly shared without explicit permission. 

## 1. Eckart–Young–Mirsky Theorem

Aditya is doing image compression with SVD for his EE 66 homework (meta!).

He's already tried using 2D DTFS to compress images, but he's curious how truncating the SVD compares to that. The **Eckart–Young–Mirsky theorem** proves truncating the SVD to be the optimal rank-$k$ approximation in squared Frobenius error. To trust but verify, Aditya wants to prove it.

Let $\mathbf{A} \in \mathbb{R}^{m\times n}$ have rank $r \le \min\{m,n\}$ and full SVD

$$\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^\top = \sum_{i=1}^{r} \sigma_i \mathbf{u}_i \mathbf{v}_i^\top, \qquad \sigma_1 \ge \cdots \ge \sigma_r > 0,$$

where $\mathbf{U} \in \mathbb{R}^{m\times m}$ and $\mathbf{V} \in \mathbb{R}^{n\times n}$ are orthonormal and $\mathbf{u}_i$, $\mathbf{v}_i$ are their $i$th columns. The terms past $i = r$ vanish because $\sigma_i = 0$ there.

For $1 \le k < r$, the *rank-k truncated SVD* keeps the first $k$ terms:

$$\mathbf{A}_k = \sum_{i=1}^{k} \sigma_i \mathbf{u}_i \mathbf{v}_i^\top.$$

Aditya chooses to define the "size" of a matrix with the squared *Frobenius norm*. This is the sum of the squared entries,

$$\|\mathbf{M}\|_F^2 = \sum_{a=1}^{m}\sum_{b=1}^{n} M_{ab}^2,$$

which is akin to putting all the entries of $\mathbf{M}$ into a vector and taking that vector's 2-norm. You may use without proof in this problem that

$$\|\mathbf{M}\|_F^2 = \sum_i \sigma_i(\mathbf{M})^2.$$

(a) Write the reconstruction error $\|\mathbf{A} - \mathbf{A}_k\|_F^2$ in terms of the singular values of $\mathbf{A}$.

(b) Now let $\mathbf{B}$ be *any* $m \times n$ matrix of rank at most $k$. We need to show that this arbitrary matrix cannot do better than $\mathbf{A}_k$.

Fix $i \in \{1,\dots,r-k\}$, let $\mathbf{E} = \mathbf{A} - \mathbf{B}$, and let $\mathbf{E}_{i-1}$ keep the first $i-1$ terms of the SVD of $\mathbf{E}$. Then

$$\sigma_i(\mathbf{E}) = \sigma_1(\mathbf{E} - \mathbf{E}_{i-1}) = \sigma_1(\mathbf{A} - (\mathbf{B} + \mathbf{E}_{i-1})).$$

Show that $\mathbf{B} + \mathbf{E}_{i-1}$ has rank at most $k + i - 1$. Conclude

$$\sigma_i(\mathbf{E}) = \sigma_i(\mathbf{A} - \mathbf{B}) \ge \sigma_{k+i}(\mathbf{A}),$$

i.e. the $i$th singular value of the error $\mathbf{E}$ introduced by subtracting any rank-$k$ matrix is at least as large as the $(k+i)$th largest singular value of $\mathbf{A}$.

*Hint: you may use without proof the fact that* $\operatorname{rank}(\mathbf{B} + \mathbf{E}_{i-1}) \le \operatorname{rank}(\mathbf{B}) + \operatorname{rank}(\mathbf{E}_{i-1})$.

*Hint: note also that* $\sigma_1(\mathbf{A} - \mathbf{D}) \ge \sigma_{\ell+1}(\mathbf{A})$ *whenever* $\operatorname{rank}(\mathbf{D}) \le \ell < r$.

(c) Write the squared error $\|\mathbf{A} - \mathbf{B}\|_F^2$ in terms of the singular values $\sigma_i(\mathbf{A} - \mathbf{B})$ of the error matrix, and use the previous part to show that

$$\|\mathbf{A} - \mathbf{B}\|_F^2 \ge \sum_{i=k+1}^{r} \sigma_i^2.$$

*Hint: Expand* $\mathbf{A} - \mathbf{B}$ *using its own SVD, as in part (a).*

(d) Let's recap. We've identified the lower bound on Frobenius error $\|\mathbf{A}-\mathbf{B}\|_F^2$, for any matrix $\mathbf{B}$ of rank at most $k$, as $\|\mathbf{A}-\mathbf{A}_k\|_F^2$. Hence every matrix of rank at most $k$ has error at least as large as the error of $\mathbf{A}_k$. This means $\mathbf{A}_k$ achieves the lowest possible error! This proves the **Eckart–Young–Mirsky theorem for the Frobenius norm**: a truncated SVD is the best approximation among all matrices of rank at most $k$.

Briefly explain what this theorem means for image compression. In your answer:

- List how many numbers you need to store $\mathbf{A}_k$ and how many to store $\mathbf{A}$, in terms of $m$, $n$, and $k$.
- Describe how you might choose $k$ in practice. What could the singular values of $\mathbf{A}$ tell you?

## 2. Pseudoinverses

This question introduces the Moore-Penrose Pseudoinverse. It generalizes matrix inverses to all matrices, regardless of dimension and rank! It also lets us solve problems we have explored in the past, such as least squares and minimum norm problems.

***Definition*** *(Compact Moore-Penrose Pseudoinverse):*

*If the compact SVD of a matrix* $\mathbf{A} \in \mathbb{R}^{m \times n}$ *is*

$$\mathbf{A} = \mathbf{U}_r \Sigma_r \mathbf{V}_r^\top \tag{1}$$

*then the compact Moore-Penrose Pseudoinverse of* $\mathbf{A}$ *is*

$$\mathbf{A}^\dagger = \mathbf{V}_r \Sigma_r^{-1} \mathbf{U}_r^\top \tag{2}$$

Recall from the compact SVD that $\mathbf{U}_r$ and $\mathbf{V}_r$ are square or tall matrices with $r = \text{rank}(\mathbf{A})$ orthonormal columns. $\Sigma_r$ is a square diagonal matrix with nonzero diagonal entries. Its inverse $\Sigma_r^{-1}$ is also diagonal, with entries being the reciprocals of those in $\Sigma_r$.

(a) i. Find expressions for $\mathbf{A}^\dagger\mathbf{A}$ and $\mathbf{A}\mathbf{A}^\dagger$ using the SVD definitions above. Simplify fully without any assumptions about $r = \text{rank}(\mathbf{A})$.

**Hint:** $\mathbf{Q}^\top\mathbf{Q} = \mathbf{I}$ for any $\mathbf{Q}$ with orthonormal columns, but $\mathbf{Q}\mathbf{Q}^\top \neq \mathbf{I}$ in general. Think about why this is, and in which particular cases we *can* say $\mathbf{Q}\mathbf{Q}^\top = \mathbf{I}$.

ii. The matrices from the previous part allow us to project a vector onto some specific space. Which space does each matrix ($\mathbf{A}^\dagger\mathbf{A}$ and $\mathbf{A}\mathbf{A}^\dagger$) project a vector onto?

**Hint:** Recall from least squares that $\mathbf{Q}(\mathbf{Q}^\top\mathbf{Q})^{-1}\mathbf{Q}^\top$ projects onto $\text{Col}(\mathbf{Q})$. What does this simplify to when $\mathbf{Q}$ has orthonormal columns?

(b) Let's examine the case where $\mathbf{A} \in \mathbb{R}^{n \times n}$ is a square, invertible matrix, so $r = n$. Show that $\mathbf{A}^\dagger = \mathbf{A}^{-1}$.

*Hint: Start from part (a), and find the shapes of* $\mathbf{U}_r$ *and* $\mathbf{V}_r$ *when* $r = n$.

So for invertible matrices, the pseudoinverse is just the inverse. Next, let's compare it to the least squares and minimum norm solutions.

(c) For a tall matrix $\mathbf{A}$ ($m > n$) with full column rank, the least squares solution to $\mathbf{A}\mathbf{x} = \mathbf{b}$ is

$$\mathbf{x}^* = (\mathbf{A}^\top\mathbf{A})^{-1}\mathbf{A}^\top\mathbf{b} \tag{3}$$

Using the SVD of $\mathbf{A}$, write $(\mathbf{A}^\top\mathbf{A})^{-1}\mathbf{A}^\top$ in terms of $\mathbf{U}_r$, $\Sigma_r$, and $\mathbf{V}_r$. State how this relates to the pseudoinverse $\mathbf{A}^\dagger$.

*Hint: Since* $\mathbf{A}$ *has full column rank,* $r = \text{rank}(\mathbf{A}) = n$*. What, then, can you say about* $\mathbf{V}_r$*?*

(d) In a minimum norm problem, we often have an underdetermined system with the same matrix equation. For a wide matrix $\mathbf{A}$ ($m < n$) with full row rank, $\mathbf{A}\mathbf{x} = \mathbf{b}$ has infinitely many solutions. The minimum norm solution, which minimizes $\|\mathbf{x}\|$, is[1]

$$\mathbf{x}^* = \mathbf{A}^\top(\mathbf{A}\mathbf{A}^\top)^{-1}\mathbf{b} \tag{4}$$

Using the SVD of $\mathbf{A}$, write $\mathbf{A}^\top(\mathbf{A}\mathbf{A}^\top)^{-1}$ in terms of $\mathbf{U}_r$, $\Sigma_r$, and $\mathbf{V}_r$. State how this relates to the pseudoinverse $\mathbf{A}^\dagger$.

*Hint: Since* $\mathbf{A}$ *has full row rank,* $r = \text{rank}(\mathbf{A}) = m$*. So* $\mathbf{U}_r$ *is a square orthonormal matrix* ($\mathbf{U}_r^{-1} = \mathbf{U}_r^\top$).

[1] You don't need to know why this is the minimum norm solution, but if you're curious, take EECS 127!

## 3. Counting Raccoons

Ryan is trying to domesticate his local raccoons. First, he needs to count them! Unfortunately, his daily measurements are unreliable: the raccoons are hard to find, and he cannot tell which ones he has already counted. To smooth out the day-to-day fluctuations, he applies a two-point moving average filter to his measurement signal $x[n]$:

$$y[n] = \frac{1}{2}(x[n] + x[n-1]), \qquad n \in \mathbb{Z}.$$

Thus, $y[n]$ is the average of today's and yesterday's measurements. This happens to be an LTI system, where the impulse response is:

$$h[n] = \frac{1}{2}\delta[n] + \frac{1}{2}\delta[n-1].$$

To apply our filter to our measurement signal, we perform an operation called a **convolution**. The output of our system is the input convolved with the filter's impulse response. We denote the convolution with $*$, so we write our output as $y[n] = x[n] * h[n]$.

For parts (a) and (b), use the measurement signal

$$x[n] = \begin{cases} 2, & n = 0, \\ 4, & n = 1, \\ 2, & n = 2, \\ 0, & \text{otherwise.} \end{cases}$$

(a) One way to determine the output that Ryan gets after applying his filter is by using the **echo method** of convolution. Let's walk through how to do that!

i. Write $x[n]$ as a sum of scaled, shifted unit impulses.

ii. For each impulse, sketch the corresponding scaled, shifted copy of $h[n]$. It may help to put each of these plots on separate axes, but stacked vertically and with the same horizontal scale.

iii. Add the shifted and scaled copies of your impulse response pointwise to obtain and sketch $y[n]$.

(b) Ryan is grumpy today because one of the raccoons stole his lunch. He no longer wants to use the echo method, and decides that he will use the **flip-and-shift** method to calculate his convolution. Here's how we express the flip-and-shift convolution mathematically:

$$y[n] = \sum_{k=-\infty}^{\infty} h[k]x[n-k].$$

Let's walk through what this means and how to calculate it on the same input as the previous part.

i. First, we want to reverse (flip) our input signal in time. This gives us $x[-k]$. Sketch $x[-k]$ and $h[k]$ as a function of $k$.

ii. Now, we want to shift our reversed input. How can we shift $x[-k]$ to obtain $x[n-k]$?

iii. To compute $y[n]$, we want our flipped-and-shifted input signal to have nonzero overlap with $h[k]$. For what values of $n$ do the two signals have nonzero overlap?

iv. For each such $n$, sketch $h[k]$ and $x[n-k]$ on the same axes, with $k$ on the horizontal axis.

v. For each of these $n$, multiply the overlapping sample values and sum the products to calculate $y[n]$. Check that your result agrees with part (a)!

(c) Ryan records a new measurement:

$$x_{\text{new}}[n] = \begin{cases} 4, & n = -1, \\ 2, & n = 0, \\ 6, & n = 2, \\ 0, & \text{otherwise.} \end{cases}$$

Determine and sketch the corresponding output $y_{\text{new}}[n]$ using a convolution method of your choice. Show your work!

(d) Kevin has also been trying to count raccoons for some time, and he's fascinated by Ryan's efforts. However, he wants to understand some general properties of Ryan's filter. Kevin considers the following two test signals, defined for all integers $n$:

$$x_{\text{const}}[n] = 4, \qquad x_{\text{alt}}[n] = 4(-1)^n.$$

**Without computing any convolution**, determine the output for each signal. Consider what the filter does to two consecutive samples. Explain your answers!

(e) Kevin introduces Ryan to the frequency response $H(\omega)$:

$$H(\omega) = \sum_{k=-\infty}^{\infty} h[k]e^{-j\omega k},$$

Here, $\omega$ is discrete-time angular frequency, and the units are radians per sample.

The frequency response answers the question, "What does my filter do to different *frequencies* present in my signal?"

Unfortunately, Kevin slipped on a banana peel left behind by a raccoon and hit his head, so he's forgotten how to calculate the frequency response. Use the given formula to find the frequency response for the two-point moving average.

(f) Kevin and Ryan want to investigate the magnitude of the frequency response, which tells them how the strength of different frequency components in Ryan's input are changed by the filter. Find a simplified expression for $|H(\omega)|$ and sketch it for $-\pi \leq \omega \leq \pi$. Label the values at $\omega = -\pi, 0, \pi$.

*Hint: Balance the exponents by factoring out $e^{-j\omega/2}$ from $1 + e^{-j\omega}$. Then apply Euler's identity.*

(g) A particularly naughty raccoon realized what Kevin and Ryan were up to and it stole their papers, so they lost their plots and can no longer interpret them. To help out, connect your magnitude plot in part (f) to the two outputs in part (d). What frequencies do the constant and alternating signals correspond to? What frequencies does this filter keep, and which does it remove? In one or two sentences, explain why this is useful for smoothing out Ryan's day-to-day measurements.

**4. Discrete One-Dimensional Edge Detector (Adapted from EE 120 Homework)**

So far, we've mostly interpreted systems as describing how a state vector changes over time. But, you can also interpret a system as a more general transformation from an input signal $x[n]$, to an output signal $y[n]$. Signal processing engineers gets to design these transformation to make $y[n]$ reveal new informations about a particular $x[n]$. In this question, consider the discrete LTI system characterized by the following relationship:

$$y[n] = x[n] - x[n-1].$$

It looks simple, but test it on a few inputs and see what it picks out.

For subparts (a)-(d) of this question, given the input signal $x[n]$, plot the corresponding output signal $y[n]$.

(a) Heaviside step function: $u : \mathbb{Z} \to \mathbb{R}$, where $\forall\, n \in \mathbb{Z}$, $u[n] = \begin{cases} 0 & \text{if } n < 0 \\ 1 & \text{if } n \geq 0 \end{cases}$

(b) Rectangular function: $x : \mathbb{Z} \to \mathbb{R}$, where $\forall\, n \in \mathbb{Z}$, $x[n] = \begin{cases} 1 & \text{if } |n| \leq 2 \\ 0 & \text{if } |n| > 2 \end{cases}$

(c) Constant function: $x : \mathbb{Z} \to \mathbb{R}$, where $\forall\, n \in \mathbb{Z}$, $x[n] = 1$

(d) Sign-alternating signal: $x : \mathbb{Z} \to \mathbb{R}$, where $\forall\, n \in \mathbb{Z}$, $x[n] = (-1)^n$

(e) Using your plots from parts (a)–(d), briefly explain why this system is aptly called a *discrete one-dimensional edge detector*. Describe where $y[n]$ is nonzero, in terms of how $x[n]$ changes. Does the system's response provide a clue as to which direction (up or down) each edge in the input signal goes? Describe what you notice.

Finally, compare this system with the two-point moving average $\frac{1}{2}(x[n] + x[n-1])$ from the Counting Raccoons problem. List which of the constant and sign-alternating signals each system removes (outputs zero for).

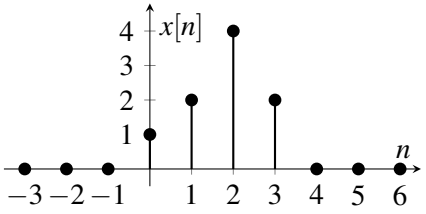
The basic concept behind our edge detector is the fundamental idea behind Convolutional Neural Networks (CNNs), used for a broad variety of image classification, localization, and processing applications.

## 5. Convolution with Toeplitz Matrices

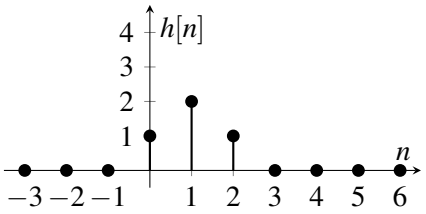
This question will introduce you to Toeplitz matrices, which let you represent convolution as matrix multiplication. Up until now, linear algebra and signals and systems may have felt like separate topics, but they are actually related in quite an elegant way!

Consider the input signal $x[n]$ and filter impulse response $h[n]$ shown below.

Input signal




Filter impulse response




Let $y[n] = x[n] * h[n]$ be the output of the filter. You can think of $x[n]$ as one row of pixel brightnesses and $h[n]$ as a blur that mixes each pixel with its neighbors (scaled by 4 to keep the arithmetic in integers).

(a) Recall the definition of discrete-time convolution,

$$x[n] * h[n] = \sum_{k=-\infty}^{\infty} x[k]h[n-k].$$

Show that convolution is commutative, or that $x[n] * h[n] = h[n] * x[n]$.
*Hint: A dummy variable $\ell$ in terms of n and k may be helpful.*

(b) As you showed in part (a), you can flip and shift either the filter or the input signal and get the same result. Since images are usually much larger than your filter, let's flip and shift the *filter*:

$$y[n] = \sum_{\ell=-\infty}^{\infty} x[\ell]h[n-\ell].$$

Use the **flip-and-shift** method to compute $y[n]$ for each shift $n = 0, 1, \ldots, 5$, writing out the multiply-and-add for each.

(c) As you've seen previously, you can store the samples of a discrete-time signal in a vector. Recall that you can represent a state-space system with a transition matrix that maps one state vector to the next. In the same way, you can represent this filter as a matrix that maps the input vector to the output vector.

Store the nonzero input samples and the output samples as

$$\mathbf{x} = \begin{bmatrix} 1 \\ 2 \\ 4 \\ 2 \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} y[0] \\ y[1] \\ y[2] \\ y[3] \\ y[4] \\ y[5] \end{bmatrix}.$$

Find the matrix $H$ such that $H\mathbf{x} = \mathbf{y}$, and give its dimensions. Each entry of $\mathbf{y}$ is the dot product of one row of $H$ with $\mathbf{x}$, so each expression from part (b) gives one row of $H$.

(d) The matrix you constructed is a **Toeplitz matrix**: its entries are constant along each diagonal running from the upper left to the lower right (if you didn't get this, check your work!).

To see why convolution produces this pattern, lets number the rows and columns of $H$ starting from 0. Row $n$ of $H$ times $\mathbf{x}$ must equal $y[n] = \sum_\ell x[\ell]h[n-\ell]$, so the entry in row $n$, column $\ell$ is $H_{n\ell} = h[n-\ell]$. This entry depends only on the difference $n-\ell$. Every entry on the same diagonal has the same $n-\ell$, so each diagonal holds a single filter value. Equivalently, each column is a copy of $h$, shifted down one row from the column before it.

Thus, discrete-time convolution for an LTI system can be represented using a Toeplitz matrix! This is really convenient for us because we have lots of fancy computational methods to compute matrix multiplications efficiently.

Note that we can also analyze this through the *echo view* of convolution. For your matrix above, notice that your input $\mathbf{x}$ is selecting $x[n]$ of the first column, which is the impulse response shifted by $n$, implying your output is a linear combination of *n-shifted impulse responses* scaled by $x[n]$.

Write out the Toeplitz matrix $H$ that applies a filter with values $h_0, h_1, h_2$ to an input signal with values $x_0, x_1, x_2, x_3, x_4$, so that $H\mathbf{x} = \mathbf{y}$ gives the full convolution. Then, try to generalize this to write out $H$ for any filter length $M$ and signal length $N$. You may find it useful to sketch the matrix using ellipses. What are the dimensions of $H$?

*Hint: How many output samples are there? What is the relationship between subsequent columns of $H$?*

**6. Mid-Semester Survey Form**

Please fill out The Mid-Semester Survey by following the link below:

Mid-Semester Feedback Form

Attach a screenshot of the confirmation page to your homework submission. We appreciate your feedback!

UCB EE 66, Fall 2026, Homework 10, All Rights Reserved. This may not be publicly shared without explicit permission. 